Homework 2 (CS59300 ASE)

Test Generation and Improvement Using Coding Agents

Weixi Tong November 2, 2026

I used Codex to write tests for Papertrail, the web application from Homework 1. I first asked it to generate a test suite, then gave it feedback in two rounds. I kept the application code the same so I could compare the tests fairly.

1. Test Generation Approach

Papertrail lets users search arXiv, save papers, upload PDFs, generate summaries, and ask questions about a paper. It uses Flask, SQLite, and pypdf, with a JavaScript frontend.

I asked Codex to read the README, application code, and existing tests, then generate tests for normal use, edge cases, and errors. My first prompt included:

> Generate meaningful, executable tests for the main functionality, including normal cases, edge cases, and error handling. Prefer the existing unittest framework unless there is a clear reason to change it.

Codex used `unittest` and the Flask test client. The initial suite had 37 test methods.

Test group	Methods	Examples
Main application functions	15	Search, saving papers, uploads, summaries, questions, editing, and deletion.
Input checks and errors	14	Invalid requests, bad PDFs, missing papers, service failures, and login checks.
Individual functions	8	Database transactions, text selection, citations, and service requests.

The tests used temporary databases and generated sample PDFs. Calls to arXiv and the model used fixed sample responses. This allowed the tests to run without network access or paid requests. The tests checked returned values, saved records, PDF files, and whether the right requests were made.

The 14 tests from Homework 1 were excluded from the main comparison. Older HW2 files were also excluded. The reported results come from the three folders `initial_stage`, `round1_stage`, and `round2_stage`. [1–4]

2. Initial Test Evaluation

Coverage and correctness

The initial run passed 35 of 37 methods. Two methods failed on three input cases. There were no test errors or skipped tests. The suite covered 474 of 522 executable lines (90.80%) and 135 of 174 branch outcomes (77.59%).

All stages measured the same four files: `app.py`, `database.py`, `pdf_tools.py`, and `providers.py`. Coverage was measured with coverage.py 7.17.0 using the same settings. Frontend code was outside this measurement.

Codex reviewed the test inputs and expected results against the README and sample data. It found no incorrect expectations, but some checks needed improvement. One timeout test started with an empty library, so it could miss an error that deleted saved papers. Another test counted saved answers without checking their contents. The PDF samples all supplied title and author information, so they did not test how well the app could find that information in the page text.

The main coverage gaps were PDF limits, PDF metadata extraction, cache expiry, redirects, and failed downloads. For example, `pdf_tools.py` had only 74.77% line coverage and 59.62% branch coverage.

Two bugs found

Bug 1: invalid analysis input causes a server error. Sending `{"kind": []}` or `{"kind": {}}` to the analysis endpoint returned HTTP 500. The expected result was HTTP 400 with an input error. The app checked the value in a set before checking its type. The detecting method in `test_validation.py` was:

```
test_invalid_analysis_payloads_are_client_errors_without_provider_calls
```

Bug 2: an unusual page number causes a server error. The search request `page=%C2%B2` contains the superscript character ². It passed `isdigit()`, but `int()` could not convert it. The app returned HTTP 500 instead of the expected input error 400. The detecting method was:

```
test_invalid_search_parameters_are_client_errors
```

The expected 400 responses follow how the app handles other invalid inputs. The recorded test output shows the actual 500 responses. Bug 1 has two failing input cases, so these two bugs produced three failures in the output. [2]

3. Iterative Test Improvement

Round 1: clearer PDF requirements

I first asked Codex to focus on PDF handling. I specified file-size, page-count, and text-length limits; PDFs without saved title or author information; and failures during optional AI metadata extraction. Part of my feedback was:

> For rejected uploads, verify that no paper record or orphaned PDF file is created.

Codex kept the original 37 methods and added eight. The new tests checked a PDF at the app's 20 MiB limit and one byte above it, PDFs with 0, 1, 150, and 151 pages, and text just below, at, and above one million characters. Rejected uploads had to leave no database record or file behind.

The new samples also tested finding a title, authors, and publication year from the PDF text. Other tests checked that a failed AI metadata request still allowed the app to save the PDF and its locally extracted information.

This round passed 42 of 45 methods. All 35 methods that passed initially still passed. Coverage rose to 505/522 lines (96.74%) and 151/174 branch outcomes (86.78%). The PDF module reached 98.20% line coverage and 88.46% branch coverage. [3]

Bug 3: part of a title is saved as an author. One sample had a two-line title followed by a separate author line. The app saved the second title line as an extra author:

```
Title:    Reliable Storage for
          Research Libraries
Authors:  Dana Tester and Evan Researcher
Expected: ['Dana Tester', 'Evan Researcher']
Actual:   ['Research Libraries', 'Dana Tester', 'Evan Researcher']
```

The detecting method in `test_pdf_round1.py` was:

```
test_multiline_prominent_title_is_not_misidentified_as_an_author
```

The wrong author appeared in both the response and the database. This was a simple, clearly labeled sample; more complicated PDF layouts were not tested. The first two bugs remained, so the suite now showed three failed methods and four failure events. [3]

Round 2: use the results to improve the remaining tests

For the second round, I pointed Codex to the round 1 coverage report and test results. The code for external services still had only 76.09% branch coverage. I asked for better tests of search caching, request timing, redirects, large downloads, and interrupted downloads. I also asked:

> Also review existing provider and error-handling assertions. Strengthen any that could pass despite incorrect results or state, and explain concrete examples.

Codex added ten methods. These checked whether search results expired after ten minutes, whether different searches reused the correct results, and whether requests waited for the required interval. A controlled clock allowed these timing tests to run without real waiting. Other tests checked downloads, bad service responses, and missing model settings.

Four existing methods also gained better checks. A timeout test now saved a paper first and verified that it survived. A download failure had to leave the record and files unchanged. A retry test checked the exact saved answer and whether it was reused. A redirect test checked the request destination.

The final suite passed 52 of 55 methods. All ten new methods and the four improved methods passed. No new bug was found. The same three application bugs still caused the same four failure events. The external-service module reached 98.46% line coverage and 95.65% branch coverage. [4]

4. Final Evaluation

The table compares the three suites on the same application code. The earlier results were kept as recorded.

Suite	Passed / total	Line coverage	Branch coverage	Bugs found
Initial	35 / 37	90.80%	77.59%	2
Round 1	42 / 45	96.74%	86.78%	3
Round 2	52 / 55	98.47%	91.95%	3

The final suite covered 514/522 lines and 160/174 branch outcomes. All stages had zero test errors and zero skipped tests. The following table shows line coverage / branch coverage for each file.

File	Initial	Round 1	Round 2
`app.py`	97.57 / 91.89	98.38 / 91.89	98.38 / 91.89
`database.py`	100 / 100	100 / 100	100 / 100
`pdf_tools.py`	74.77 / 59.62	98.20 / 88.46	98.20 / 88.46
`providers.py`	89.23 / 73.91	91.54 / 76.09	98.46 / 95.65

All values are percentages.

The first round improved PDF coverage and found a new bug. The second round improved service handling and checked saved data more carefully, but found no additional bug. Some stronger checks did not increase coverage because the earlier tests already ran those lines.

The final review found no incorrect expected results. Coverage shows how much code ran; it does not show that every result was checked correctly. For example, a saved answer can exist but contain the wrong text. Also, the higher final pass rate came from adding passing tests, not from fixing the app. The three known bugs were left in place for the comparison.

5. Reflection

Codex understood the main application functions and wrote useful tests for saving papers, uploading PDFs, and handling summaries and questions. It was also good at creating sample data and checking database and file contents.

Its first version missed less obvious cases. The PDF samples were too simple, some error tests did not protect existing data, and an immediate repeat of a search did not test cache expiry. Clear examples and actual test results helped it address these gaps. Asking for a specific behavior was more useful than simply asking for more tests.

I learned to review what each test checks, even when it passes. A test that counts answers is weaker than one that checks the answer itself. I would use the same approach again: keep the first result, give focused feedback, and compare both coverage and the checks made by the tests.

These tests do not cover browser interaction, real service availability, actual model answer quality, or all PDF layouts. They also do not establish how the app behaves during simultaneous requests or a crash. Those limits remain despite the high backend coverage.

Codex helped prepare the prompts, tests, and report. I directed the two improvement rounds. I did not manually edit the tests or fix the application during this experiment.

6. GitHub Repository

Repository: `https://github.com/VichyTong/CS593-ASE`.

The final tests are in `solutions/hw2/round2_stage/tests/`. The PDF report and LaTeX source are in `solutions/hw2/docs/`. The HW2 README explains the files and recorded results.

Install and run

From the repository root, use Python 3.11 or newer:

```
python3 -m venv /tmp/papertrail-hw2-venv
TASK_PY=/tmp/papertrail-hw2-venv/bin/python
"$TASK_PY" -m pip install -r \
  solutions/hw2/round2_stage/requirements-test.txt
"$TASK_PY" -B solutions/hw2/round2_stage/run_round2.py \
  --output /tmp/papertrail-hw2-final-run
```

Use a new output directory each time. The runner saves test output, `results.json`, `coverage.json`, and an HTML coverage report. It checks that the recorded application and earlier test files are still present and unchanged, so keep the three stage folders together.

On the recorded application, the expected result is 52 passed methods, three failed methods, and four failure events. The command returns exit code 1 because the three known bugs are still present. The tests use temporary data and fixed service responses, so no API key is needed.

Supporting records

The quoted prompts, test results, and coverage reports are saved here:

[1] `solutions/hw1/README.md`: application requirements.

[2] `solutions/hw2/initial_stage/`: initial request, review notes, and `evidence/initial-only/`.

[3] `solutions/hw2/round1_stage/`: first feedback, review notes, and `evidence/round1-only/`.

[4] `solutions/hw2/round2_stage/`: second feedback, review notes, `evidence/round2-only/`, and `evidence/comparison.json`.